ISA Design

Overview

We are designing an ISA for a 10-bit single-cycle CPU capable of performing simple arithmetic and logic operations, condition-based branching, and loops. The processor uses a RISC Load-Store architecture with separate instructions and data memory, where the register width, bus width, instruction length, and ALU width are all 10 bits. We decided on a 2-operand architecture that uses eight registers (R0 hardwired to zero and seven general-purpose registers). Our design also utilizes two flip-flops to store hardware flags and two special-purpose registers to buffer our inputs and outputs. The 10-bit fixed-length instructions are divided into three formats: Register (R-type), Immediate (I-type), and Branch/Jump (B/J-type). Justifications for our design choices are provided in the "Tradeoffs and Justification" section.

Number of Operands

Our ISA uses a 2-operand architecture. Each R-type and I-type instruction names two operands: a destination register (Rd), which also supplies the first source value, and a second source (Rs or Imm). For example, ADD R1, R2 computes R1 = R1 + R2, so the original value of R1 is overwritten by the result.

There are three exceptions. CMP reads two registers (its Rd and Rs fields) but has no destination, because its result is stored in the E and L flip-flops instead of a register. The B/J-type instructions (BEQ, BLT, and JMP) have a single operand, the 6-bit offset. Finally, the IO instruction uses one register (Rd) and an Imm control flag, so it is a single-register operation.

Types of Operands

The ISA supports five types of operands, summarized in the table below.

Operand Type	Location	Size / Range	Used By
Register	Eight registers, R0 hardwired to zero, seven general-purpose.	3-bit register field	ADD, SUB, AND, OR, XOR, SLL, SRL, CMP, LW, SW; Rd of LDI, ADDI, IO

Immediate	Imm field inside the instruction	3 bits: 0 to 7 (LDI, zero-extended), -4 to +3 (ADDI, sign-extended)	LDI, ADDI
Memory	Data memory, addressed through the register in Rs	10-bit address held in a register	LW, SW
PC-relative offset	Offset field inside the instruction, added to PC + 1	6 bits, signed: -32 to +31	BEQ, BLT, JMP
I/O	Input Buffer Register (IBR) and Output Buffer Register (OBR)	10-bit buffers, direction chosen by Imm[0]	IO

Number of operations

The ISA has **16 operations**. This is the maximum a 4-bit opcode can encode, so every opcode from 0000 to 1111 is used. This number covers all three benchmark categories (arithmetic and logic, condition checking, loops) and the input/output requirement. A 3-bit opcode would allow only 8 operations, which is too few for that functionality.

Types of operations

The 16 operations fall into seven categories:

Category	Instructions	What they do
Arithmetic (3)	ADD, SUB, ADDI	Addition and subtraction. ADDI also handles increment and decrement.
Logical (3)	AND, OR, XOR	Bitwise operations on two registers
Shift (2)	SLL, SRL	Logical shifts left and right by 0 to 15 bits (10 or more gives 0)
Data transfer (3)	LW, SW, LDI	Move data between registers and data memory, and load a small constant
Comparison (1)	CMP	Compares two registers and sets the E and L flags

Control flow (3)	BEQ, BLT, JMP	Conditional and unconditional branching using PC-relative addressing
Input/Output (1)	IO	Reads the input buffer into a register, or writes a register to the output buffer

Instruction Set

In the table below, we used conventional assignment notation. Some noteworthy considerations about our design:

- Immediate and offset values have to be sign-extended if they are signed (as in the ADDI and B/J-type instructions) or zero-extended if they are unsigned (as in the LDI instruction) to be stored and processed.
- We use a single I-type IO instruction. Imm[0] selects the direction. If Imm[0] = 0, the value in the IBR is loaded into Rd. If Imm[0] = 1, the value in Rd is loaded into the OBR. Imm[2:1] are reserved and should be set to 0.
- The shift amount is 4 bits (0 to 15). Since a 10-bit register is fully shifted out at 10 or more, those amounts produce 0. As our R-type instructions lack a 4-bit field, the shift amount is taken from the lower 4 bits of the value held in register Rs (written Rs[3:0]); the remaining bits of that value are ignored.
- Due to the limited fields of a 2-operand architecture, we are unable to perform condition checks and branching in a single instruction. To get around this, we use the CMP instruction to perform equal-to and less-than condition checks simultaneously to update the E and L flip-flops, respectively. Only CMP instructions can change their values, and the values persist until the next CMP instruction. BEQ and BLT are then used to branch conditionally based on the flip-flop values. CMP treats both operands as signed 10-bit two's-complement values. L is set when Rd < Rs, computed as L = (MSB of Rd − Rs) XOR (overflow of that subtraction).

Opcode	**Mnemonic**	**Type**	**Syntax**	**Example**	**Meaning**
0000	ADD	R-type	ADD Rd, Rs	ADD R1, R2	Rd = Rd + Rs
0001	SUB	R-type	SUB Rd, Rs	SUB R1, R2	Rd = Rd - Rs
0010	AND	R-type	AND Rd, Rs	AND R1, R2	Rd = Rd & Rs (Bitwise AND)

0011	OR	R-type	OR Rd, Rs	OR R1, R2	Rd = Rd | Rs (Bitwise OR)
0100	XOR	R-type	XOR Rd, Rs	XOR R1, R2	Rd = Rd ^ Rs (Bitwise XOR)
0101	SLL	R-type	SLL Rd, Rs	SLL R1, R2	Rd = Rd << Rs[3:0] (Logical Shift Left)
0110	SRL	R-type	SRL Rd, Rs	SRL R1, R2	Rd = Rd >> Rs[3:0] (Logical Shift Right)
0111	LW	R-type	LW Rd, Rs	LW R1, R7	Rd = Mem[Rs]
1000	SW	R-type	SW Rd, Rs	SW R1, R7	Mem[Rs] = Rd
1001	LDI	I-type	LDI Rd, Imm	LDI R1, 5	Rd = ZeroExt(Imm) (Unsigned 0 to 7)
1010	ADDI	I-type	ADDI Rd, Imm	ADDI R1, -1	Rd = Rd + SignExt(Imm) (Signed -4 to +3)
1011	CMP	R-type	CMP Rd, Rs	CMP R1, R2	E = (Rd == Rs), L = (Rd < Rs)
1100	BEQ	B/J-type	BEQ Offset	BEQ 4	if (E == 1) PC = PC + 1 + SignExt(Offset)
1101	BLT	B/J-type	BLT Offset	BLT -3	if (L == 1) PC = PC + 1 + SignExt(Offset)
1110	JMP	B/J-type	JMP Offset	JMP 8	PC = PC + 1 + SignExt(Offset)
1111	IO	I-type	IO Rd, Imm	IO R1, 1	if (Imm[0] == 0) Rd = IBR else OBR = Rd

Instruction Formats

We use three distinct 10-bit formats to implement different instructions. Each field name has a meaning:

- Opcode - 4-bit operation code for identifying an instruction
- Rd - destination register and primary operand source register
- Rs - secondary operand source register (also used as memory address pointer)
- Imm - immediate value (also used as IO control flag)
- Offset - immediate value for relative addressing

R-type

Opcode (Bits 9-6) 4 bits	Rd (Bits 5-3) 3 bits	Rs (Bits 2-0) 3 bits

I-type

Opcode (Bits 9-6) 4 bits	Rd (Bits 5-3) 3 bits	Imm (Bits 2-0) 3 bits

B/J-type

Opcode (Bits 9-6) 4 bits	Offset (Bits 5-0) 6 bits

Registers

Eight 10-bit Registers (R0 hardwired to zero, R1–R7 general-purpose)

Number	Binary Encoding	Name	Use
0	000	R0	Hardwired to 0, writes are ignored
1	001	R1	Storing saved data
2	010	R2	Storing saved data
3	011	R3	Storing saved data

4	100	R4	Storing temporary data
5	101	R5	Storing temporary data
6	110	R6	Storing temporary data
7	111	R7	Storing temporary data

Special Registers and Flip Flops

Name	Use
Equality Flip Flop (E)	Stores a 1-bit zero output of the ALU during the CMP instruction
Less-Than Flip Flop (L)	Stores (MSB of the ALU subtraction result) XOR (overflow flag) during the CMP instruction, so L = 1 exactly when Rd < Rs as signed values.
Input Buffer Register (IBR)	10-bit register that holds the most recent value entered on the keyboard. An external keyboard encoder writes the value into the IBR when the user presses Enter. `IO Rd, 0` copies the current IBR value into Rd.
Output Buffer Register (OBR)	10-bit register that holds the last value written by `IO Rd, 1`. It drives the display unit: an external binary-to-BCD converter and 7-segment decoder turn the 10-bit value into digits, so a 10-bit value (up to 1023 unsigned) needs four 7-segment digits.

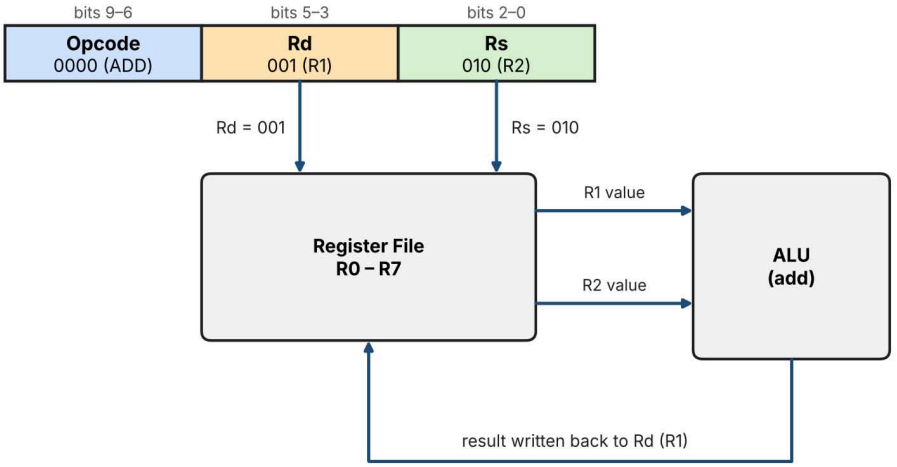
Addressing Modes

We use the following addressing modes in our ISA:

- **Register Addressing** - operands are stored in general-purpose registers. This is used by all core arithmetic, logical, and shift operations, as well as by the CMP instruction.
- **Immediate Addressing** - operands are embedded inside the encoded instruction (specifically in the Imm field of I-type instructions). This is used in all I-type instructions.
- **Register Indirect Addressing** - a pointer to the actual address in data memory is stored in a general-purpose register. This is used in the LW and SW instructions.
- **PC-Relative Addressing** - the next address in instruction memory is calculated by adding an offset to the program counter + 1. This is used in B/J type instructions.

Register Addressing

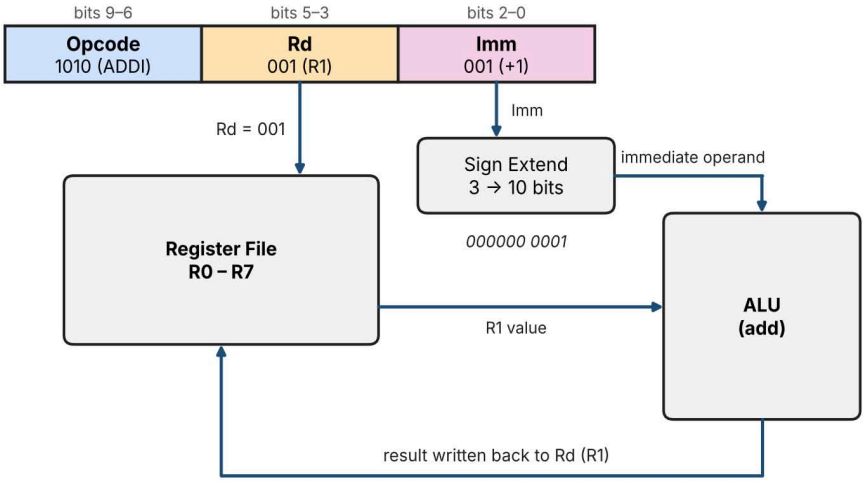
ADD R1, R2 (R-type) R1 = R1 + R2



Both operands are read from general-purpose registers. No memory access.

Immediate Addressing

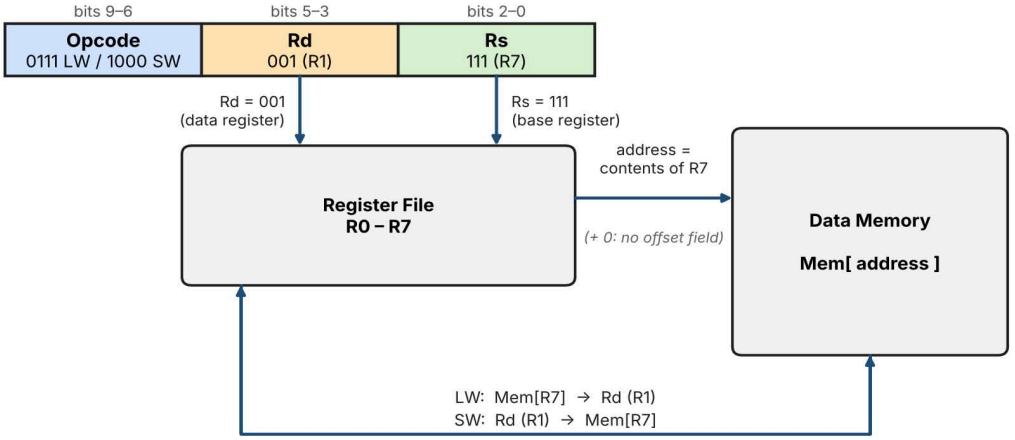
ADDI R1, 1 (I-type) R1 = R1 + SignExt(Imm)



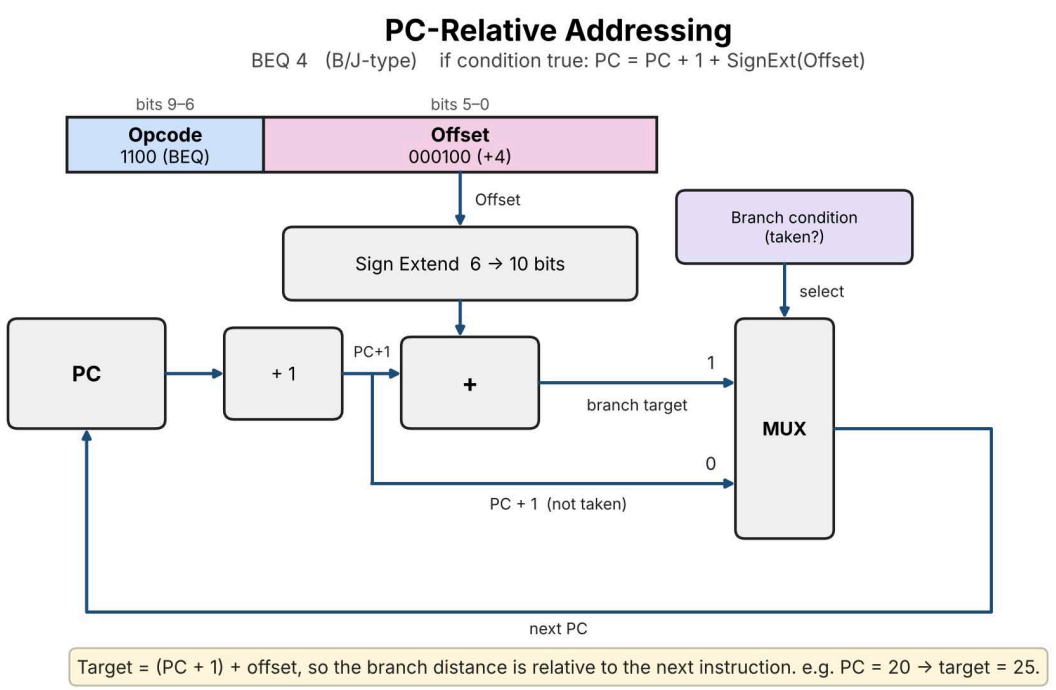
One operand is a constant stored inside the instruction itself (3-bit signed, −4 to 3).

Base (Register-Indirect) Addressing

LW R1, R7: R1 = Mem[R7] SW R1, R7: Mem[R7] = R1 (R-type)



The base register holds the data-memory address (displacement = 0). LW reads memory, SW writes it.



Benchmark Program Implementations

Benchmark A — Arithmetic operation

C Program

int x = (a + b) - c;

Register mapping

Register	Value
R1	a, then final result x
R2	b
R3	c

Assembly program

Assuming a, b, and c are preloaded into registers R1, R2, and R3, respectively, and that a does not need to be preserved

Address	Instruction	Comment
0	ADD R1, R2	R1 = a + b

1	SUB R1, R3	R1 = (a + b) - c
2	JMP -1	Stop by looping indefinitely

Benchmark B — Conditional selection

C program

```
if (a < b)
   x = a;
else
   x = b;
```

Register mapping

Register	Value
R1	a
R2	b
R3	x (initially nothing)

Assembly program

Address	Instruction	Comment
0	CMP R1, R2	L = a < b (also E = a == b)
1	BLT 3	if a < b, branch to 1 + 1 + 3 = 5
2	SUB R3, R3	R3 = 0
3	ADD R3, R2	R3 = b

4	JMP 2	jump to 4 + 1 + 2 = 7
5	SUB R3, R3	R3 = 0
6	ADD R3, R1	R3 = a
7	JMP -1	Stop by looping indefinitely

Benchmark C — Sum using a loop

C program

```
int sum = 0;

for (int i = 0; i < 10; i++) {

    sum += i;

}
```

Register mapping

Register	Value
R1	sum
R2	i
R3	loop limit 10

Assembly program

Address	Instruction	Comment
0	SUB R1, R1	sum = 0
1	SUB R2, R2	i = 0
2	LDI R3, 7	R3 = 7 (maximum immediate value)
3	ADDI R3, 3	R3 = 10 (make the loop limit)
4	CMP R2, R3	E = i == 10 (and L = i < 10)

5	BEQ 3	if i == 10, branch to 5 + 1 + 3 = 9
6	ADD R1, R2	sum = sum + i
7	ADDI R2, 1	i = i + 1
8	JMP -5	jump to 8 + 1 - 5 = 4
9	JMP -1	Stop by looping indefinitely

Tradeoffs and Justification

1. Eight Registers vs. Immediate Width

Trade-off: We chose eight registers (seven general-purpose and R0 = 0) , which require 3 bits to identify each register. This leaves only 3 bits for the immediate value in our I-type instruction format.

Justification: Seven usable registers provide enough storage for simple programs while keeping the instruction length fixed at 10 bits. However, the 3-bit immediate restricts constants to a small range, so larger constants may require multiple instructions.

2. Unified I/O Instruction vs. Dedicated Opcodes

Trade-off: We use a single IO instruction for both input and output instead of separate IN and OUT instructions. The direction is selected by Imm[0]: IO Rd, 0 reads the input buffer (IBR) into Rd, and IO Rd, 1 writes Rd to the output buffer (OBR).

Justification**:** Our ISA has 16 operations, which is exactly the number a 4-bit opcode can encode. Separate IN and OUT instructions would make 17 operations, which does not fit in 4 bits. Since the ISA must be general-purpose, each existing operation covers a capability we need (arithmetic, logic, shifts, data memory access, comparison, branching), so we could not drop one to make room. A unified IO instruction therefore lets all required functionality fit within the fixed 10-bit format. The cost is that the direction is encoded in the Imm field rather than in the opcode. This makes the instruction less readable in assembly and requires the decoder to check Imm[0] before selecting the data path. Because the Imm field is otherwise unused by IO, this cost is small: the bit is decoded only for this one opcode.

3. Infinite Loop (JMP -1) vs. Dedicated HALT Opcode

Trade-off: We stop a program with JMP -1, which jumps to its own address, instead of reserving an opcode for HALT. Once the program is finished, the CPU keeps fetching and executing this jump, and the PC never leaves that address.

Justification: Our ISA uses all 16 opcodes, the maximum a 4-bit opcode can encode, so a HALT opcode would require removing an existing operation. Every operation provides a capability a general-purpose ISA needs, such as a logic operation, data memory access or I/O, even though the three sample benchmarks exercise only some of them, so we chose a software convention over giving up a capability. Using JMP -1 as the halt convention keeps the full instruction set without changing the format. It also needs no extra hardware: the existing PC-relative branch logic already produces the self-loop. The cost is that a halted CPU is not truly idle. It keeps cycling and drawing power, and a simulator has no clean signal that the program has ended. A testbench can detect the halt by checking whether the PC repeats, which mitigates this for simulation but not for real hardware.

4. Hardware Flags vs. General-Purpose Registers

Trade-off: We use dedicated Equality (E) and Less-Than (L) flip-flops instead of storing comparison results in general-purpose registers.

Justification: This simplifies conditional branching because CMP updates the flags and BEQ or BLT checks them. However, it introduces additional hardware state and requires comparisons and branches to be performed in separate instructions.

5. Two-Operand Architecture vs. Three-Operand Architecture

Trade-off: Our CPU uses destructive two-operand instructions, such as ADD R1, R2, where the result overwrites R1.

Justification: This reduces the number of bits required to encode register operands, allowing us to fit instructions into 10 bits. However, the original value in the destination register is lost, and some calculations require additional instructions to preserve the values.

6. Fixed 10-bit Instructions vs. Larger Instruction Fields

Trade-off: All instructions have a fixed length of 10 bits, even when an operation needs more information.

Justification: Fixed-length instructions simplify instruction fetching and decoding in our single-cycle CPU. However, the limited instruction width forces compromises, such as small immediate values and a 6-bit branch offset.

7. Limited Immediate and Branch Offset Range

Trade-off: Our CPU uses a 3-bit immediate field for I-type instructions and a 6-bit offset field for B/J-type instructions. This limits the range of constants and branch/jump distances that can be represented directly in a single instruction.

Justification: The 3-bit immediate field allows LDI to load unsigned values from 0 to 7, while ADDI supports signed values from −4 to +3. In contrast, the 6-bit signed offset allows BEQ, BLT, and JMP to represent offsets from −32 to +31. These smaller fields help maintain the fixed 10-bit instruction length, simplifying instruction decoding and CPU design. However, larger constants or branch distances may require additional instructions or alternative program arrangements.

8. Shift Amount in a Register vs. in an Imm Field

Trade-off: SLL and SRL determine shift amount from Rs[3:0] instead of an immediate value. This means that the shift amount has to be loaded into a register. If LDI is used to do this, a further ADDI instruction is required if the desired shift amount is larger than 7. So a single shift operation may require up to 3 lines of assembly.

Justification: Shift amounts up to 15 require 4 bits. If we changed SLL and SRL to I-type instructions, the 3-bit Imm field would only allow us to store a maximum shift amount of 7. As our instruction formats do not have enough bits for an opcode, a target register, and a 4-bit immediate value, we have chosen to use a register to store the shift amount. LDI plus ADDI can build any shift amount up to 10, which is enough because an amount of 10 or more clears the register.

9. PC-Relative Branching vs. Register Branching

Trade-off: BEQ, BLT, and JMP use a 6-bit signed offset, so they can only reach instructions from 31 before to 32 after the current instruction (target = PC + 1 + offset). The alternative was register branching, where the target address is read from a register (e.g., JR Rd), which can reach any address in instruction memory.

Justification: A 6-bit offset is the largest that fits beside the 4-bit opcode in a 10-bit instruction, and it covers all the loops and conditionals in our benchmarks. We rejected register branching for three reasons. First, the target address would have to be built in a register before every branch, and since LDI only loads 0 to 7, building even a moderate address takes several instructions and ties up a register, which makes every loop and if-statement slower and longer. Second, all 16 opcodes are already in use, so adding a register-jump instruction such as JR would require removing an existing operation. Third, PC-relative code does not depend on where it is loaded. The cost is that PC-relative branching needs an extra adder and sign-extender in the PC-update path, and any target more than 31 instructions before or 32 after the branch needs a chain of JMP instructions. We also have no indirect jump, so function-style returns are not supported.

10. I/O Without Handshaking

Trade-off: The IO instruction has no status flag or busy signal. It reads the IBR and writes the OBR immediately, in one cycle, and the CPU cannot tell whether a new key has been entered since the last read.

Justification**:** A status bit or a stall signal would need another flag, a polling instruction, or extra control hardware, and we have no free opcode for polling. We therefore assume the keyboard encoder holds a valid value in the IBR before the program runs `IO Rd, 0`, such as a value entered before starting execution. The cost is that a program cannot wait for input or detect new input, and reading too early returns the previous value. A real system would add a ready flag or a stall line.